

## **Claims Risk in Insurance of Automobiles**

Machine Learning Project

By

Andreea-Larisa Rosu ([rosu@itu.dk](mailto:rosu@itu.dk)), Daniil Strelan ([dast@itu.dk](mailto:dast@itu.dk))

**Course:** Machine Learning (Autumn 2025)

**Course Code:** BSMALEA1KU

**Course Manager:** Jonas Juul

15<sup>th</sup> of December 2025

# 1. Introduction

Modeling insurance risks in the context of automobiles is a crucial task that allows companies make sense of the driving habits of their clients. Using data analytics and statistical methods, one can evaluate the risks and likelihood of potential damages tied to specific groups of drivers or vehicles, to accurately price policies and ensure financial viability. [1]

Therefore, the objective of this project is to investigate different machine learning model approaches. We make use of them to compute claims risk in automobile insurance using a dataset of insurance policies and their observed claim counts over a fixed exposure period.

## 1.1 Target variable

We define **claims risk** as the **expected number of claims per unit exposure**, or **claim frequency**, denoted by the formula  $\text{ClaimNB} / \text{Exposure}$ , where:

- It aims to deduce how frequently claims occur, considering the time at risk;
- We take into consideration Exposure rather than solely ClaimNB to reduce/avoid bias (e.g. specific policies with longer exposure would appear riskier, even if their risk profile is the same)
- It is a continuous, non-negative variable commonly used in risk assessment, ensuring accuracy and robustness in predictions, especially when analyzed alongside other risk factors (features) such as Age, Area, Vehicle Power, etc. [2]

Given our target variable, the task we are working with proves to be a regression problem, since there are no class predictions to compute. Therefore, our choice of metrics to evaluate the performance of the models is the Mean Squared Error (MSE), or more specifically, its squared root counterpart (RMSE).

MSE is a standard metric for regression problems, suitable for all the models we are training throughout the project. It is easy to interpret, penalizes large prediction errors more strongly, and allows us to compare our chosen models in an accurate manner. We compute the RMSE (same units as our target variable) on both the training and testing data to give us a better view of potential overfitting that may happen, as well as how well the models generalize on new and unseen data.

## 2. Dataset & Data Analysis

Our dataset consists of 678015 entries and is split into one training (80%) and one testing (20%) dataset. The data contains all related information about vehicular insurance policies, as well as claim history throughout the past year. It is divided into 12 columns, that we will use as features throughout the analysis and modeling of our tasks:

- Target:
  - ClaimNB: number of claims
  - Exposure: time at risk in years
- Features:
  - Vehicle: VehAge, VehPower, VehBrand, VehGas

- Driver: DrivAge, BonusMalus
- Location: Area, Density, Region
- Unique identifiers: IDPol

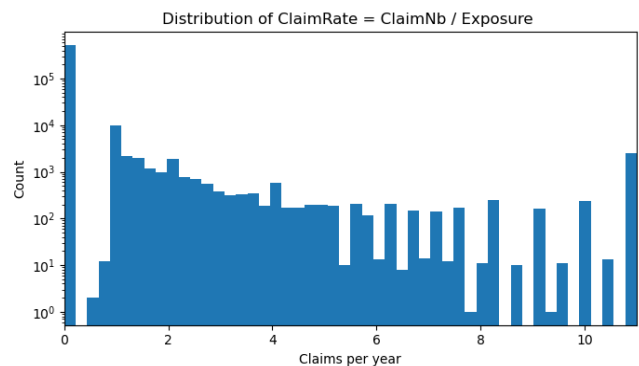
## 2.1 Data Cleaning

The dataset required only limited cleaning prior to analysis. No missing values were detected, and all variables were within reasonable and expected ranges. The policy identifier (IDpol) was removed, as it does not carry predictive information. Additionally, derived variables such as ClaimRate were excluded from modeling to avoid target leakage, since they are directly computed from the response variable. Categorical variables such as Area, VehBrand, VehGas, and Region were retained, as they potentially capture important structural and geographical differences in claim behavior.

## 2.2 Exploratory Data Analysis (EDA)

Initial descriptive statistics revealed that the average number of claims per policy is very low, with a mean value of approximately 0.05 which means that more than 90% of policies have 0 claims. This confirms that the dataset is strongly zero-inflated. The exposure variable showed substantial variation, indicating that many policies are observed for less than a full year, which was accounted for during modeling.

To better understand the relationship between claims and exposure, the variable ClaimRate was introduced, defined as the number of claims divided by the exposure period. The motivation for constructing this feature was not to use it directly for prediction, but rather to normalize claim counts by time at risk and gain exploratory insight into relative claim frequency. As expected, ClaimRate is strongly correlated with ClaimNb (correlation  $\approx 0.26$ ). This relationship is mechanical, since ClaimRate is directly derived from ClaimNb, and therefore does not provide independent information about claim risk. For this reason, ClaimRate was excluded from all predictive models to avoid target leakage. Its primary role was exploratory, allowing us to reason claim intensity independently of exposure length.



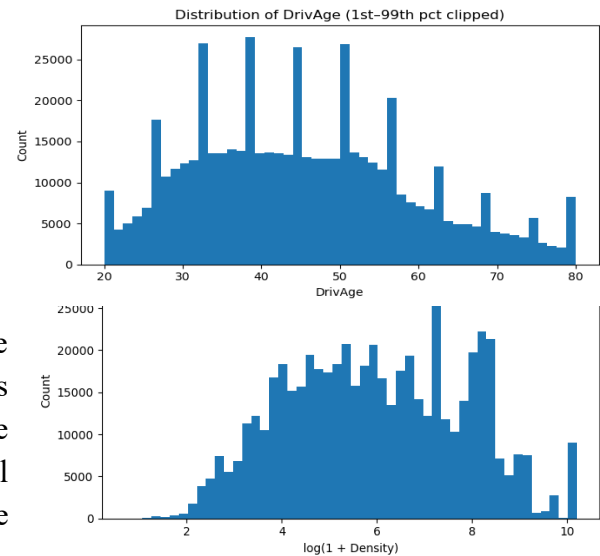
Exposure itself shows a small positive correlation with claim counts, reflecting the simple fact that policies observed for longer periods have more opportunity to experience claims. Exposure should therefore be interpreted as a scaling factor rather than a true predictor, motivating its use as an offset or weighting variable in later models.

The most meaningful feature identified during exploratory analysis was BonusMalus. Although its linear correlation with ClaimNb is modest, it captures historical driving behavior and therefore contains concentrated risk information. In the context of highly imbalanced insurance data, even small correlations are informative, and BonusMalus consistently emerged as a key variable.

Other features such as vehicle age, driver age, population density, and vehicle power exhibited very low linear correlations with claim counts. These low values do not imply irrelevance, but rather reflect the limitations of linear correlation in capturing nonlinear effects.

Drivers age has U-shaped relationship with risk with peaks at certain values that don't fall into normal distribution but were kept because of their repetitive occurrence, not skewing overall statistics and still might be valid datapoints.

To better understand the distribution of population density, a logarithmic transformation of the variable was applied and visualized. The raw density values are heavily right-skewed, with a small number of policies located in extremely dense urban areas. Applying the transformation  $\log(1 + \text{Density})$  compresses the long tail and reveals the underlying structure of the data more clearly.



The resulting distribution shows substantial variation across rural and urban environments, supporting the interpretation of density as an important but nonlinear factor influencing claim risk.

## 2.3 Principal Component Analysis (PCA)

To explore a little more in-depth the structure of our data and potential relationships between features, we implemented Principal Component Analysis [3] to a subset of 6 numeric features, specifically Exposure, VehPower, VehAge, DrivAge, BonusMalus, Density. The variables BonusMalus and Density have shown skewness during EDA (small number of policies have very large values), therefore we chose to log-transform them to ensure the variance is stabilized, as we didn't want extreme observations to influence our PCA results.

Similarly, another preprocessing step we took was standardizing all the aforementioned numerical features. Since PCA looks for directions in data where values vary the most, shifting all features to have a mean of 0 and variance of 1 ensures that all features contribute equally and fairly to the analysis.

After performing PCA on the preprocessed data, we computed the variance of the principal components. We chose the number of PC to be 2, meaning a 2-dimensional structure, as it can more easily be visualized. The total variance ratio we got was 47%, meaning that the first 2 principal components account for a bit less than half of our dataset, with the first PC being 27.5% and the second 20.2%. Although a substantial amount of information is retained, it also means that the data we are working with is very high-dimensional and requires more than 2 components to be fully represented. Any subsequent PC we added explained less than 20% of the variance each, adding up to 100% eventually, but we decided to work with the 2-dimensional PCA for the intended visualization purposes.

Figure 4 shows a projection of Policies on the principal components, colored by the occurrence of claims. We can see the imbalance in number between claim and no-claim, and policies with at least one claim are dispersed throughout the whole feature space and overlap with the non-claim policies. Due to this overlap, we can deduce that our data is not linear, meaning that our claims are influenced by many other features interacting together.

Figure 5 displays the same projection but in relation to each Area present in the dataset (categorical value). Unlike claim occurrence, area categories show a clearer separation between each other along the PCs. This means that our principal components capture geographical-related structures within the data, where more urban areas tend to cluster in specific regions, while rural areas occupy other regions. “Area” seems to be one of the more important features to keep in mind when developing our models.

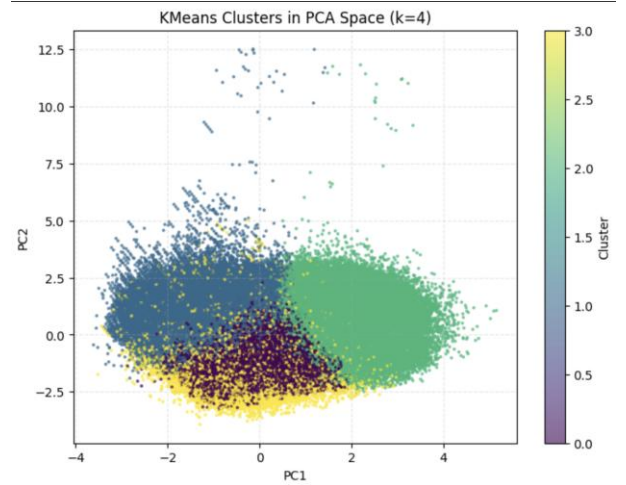
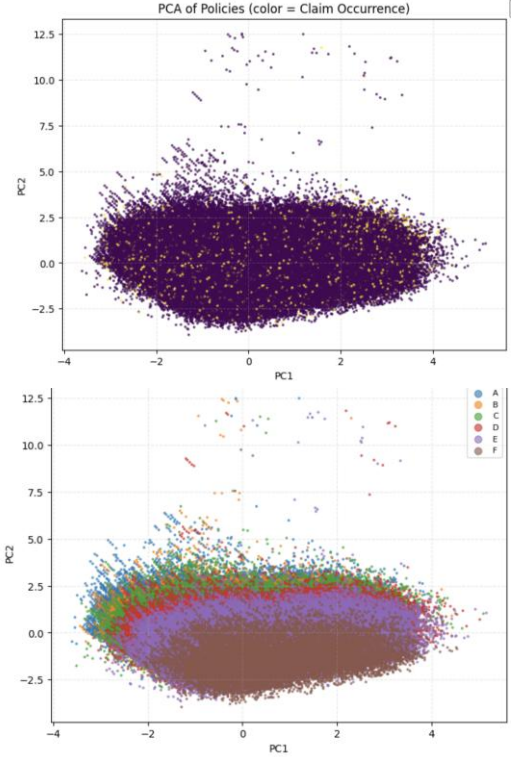
## 2.4 Clustering Method (K-Means)

To investigate the structure of our data even further, we implemented the K-means clustering method to aid in the visualization of our dataset. We chose to do this on the PCA space so we can reduce noise and the original correlation between some variables, making our k-means more stable, interpretable, and easier to visualize. We applied the k-means to the already standardized values, to avoid any scale difference complications.

We selected our k number of clusters to be 4, which provided a good visual separation in our PCA space.

We can see a partial overlap between our clusters, but also clear separation along the first principal component. This is enough to deduce that there are clear differences in claim frequency and policy characteristics, marking the presence of different risk segments.

A cluster-level summary has also been implemented to see differences between clusters and the data within them. We see that cluster 0 and 2 contain the highest claim frequencies (0.152; 0.154), while cluster 1 shows a considerably lower one (0.069). Cluster 3 has the highest BonusMalus (83.570), while cluster 2 contains the highest mean DrivAge (51.209), etc. These patterns suggest that the clusters each capture meaningful amounts of variation within the structure of the data.



### 3. Methodology

To model our claim risks, we relied on three different Machine Learning algorithms that were the most suitable for our task: Decision Tree, Feedforward Neural Network, and Poisson Regression. We will walk through every step we took to reach our results:

#### 3.1 Decision Tree (M1)

##### 3.1.1 Model Setup

The first model we implemented is the Decision Tree. We made it from scratch with the use of Python and NumPy only. Starting off with choosing our datasets, we aimed for the clean ones following the EDA we computed, “claims\_train\_clean.csv” and “claims\_test\_clean.csv”. Out of this data, we selected six numeric input features (Exposure, VehPower, VehAge, DrivAge, BonusMalus, Density) and one categorical feature (Area).

Because Area is an ordered category (from A to F, rural to urban), we encoded it as integers using a mapping built from the training set only (e.g. A->0, B->1, ..., F->5).

Our prediction target is **claim frequency**:  $y = \text{ClaimNB} / (\text{Exposure} + \epsilon)$ , where  $\epsilon$  is a small number to avoid division by 0.

We trained the tree on  $(X_{\text{train}}, y_{\text{train}})$  and evaluated it on  $(X_{\text{test}}, y_{\text{test}})$  using Root Mean Squared Error (RMSE) on claim frequency.

##### 3.1.2 Implementation

We implemented a CART-style regression tree using NumPy. At each node we tried splits of the form  $xf \leq t$ , where  $f$  is a feature index, and  $t$  is a threshold. The chosen thresholds were the midpoints between sorted unique feature values.

For each split we computed a loss based on how “spread out the target is” in each child (sum of squared errors in left + right groups). Each of the tree nodes stores: its own prediction (mean target value inside the node), the split feature and threshold, and indices of the left and right child nodes. We gave our model 3 hyperparameters, also working as stopping rules: `max_depth`, `min_samples_split`, and `min_samples_leaf`.

##### 3.1.3 Hyperparameter Tuning

To fine tune our hyperparameters, we made use of our secondary reference model from scikit-learn that we used to compare results, since it is faster and more reliable. We tested multiple settings on it through grid search, with our tested ranges being:

- `depth_values` - 4, 6, 8, 10
- `min_split_values` - 20, 50, 100
- `min_leaf_values` - 10, 20, 50

The values `max_depth` - 6, `min_samples_split` - 100 and `min_samples_leaf` - 20 were chosen and inserted into both manually and library implemented models.

## 3.2 Feedforward Neural Network (M2)

### 3.2.1 Model Setup

Using only standard libraries, we created a Feedforward Neural Network regressor from scratch. We trained it to predict the same claim frequency target ( $\text{ClaimNB} / (\text{Exposure} + \epsilon)$ ) and we used the same cleaned datasets as for the Decision Tree.

The features we used were all exclusively numeric (Exposure, Vehpower, VehAge, DrivAge, BonusMalus, Density), due to the nature of the NN requiring only numeric inputs or encoding to perform computations.

Before training, we standardized features using training-set statistics  $((x_{\text{train}} - \text{mean}) / \text{std})$  and applied it to the testing too, because gradient descent struggles if one input feature is much larger than the others. To store the input data, we create a matrix in which features correspond to rows, and samples correspond to columns. We do this because this implementation simplifies the implementation of vectorized forward and backward propagation using NumPy.

### 3.2.2 Implementation

Our model consists of one hidden layer with ReLU activation, and a linear output layer. ReLU activation function was chosen because it introduces nonlinearity while being simple and effective in our task. The same goes for linear activation, exceling in regression tasks that predict continuous targets, or, in our case, claim frequency.

We initialized weights with small random values, while biases started off at zero. Passing into the hidden layer (forward pass/propagation), we follow the formula  $Z = WA + b$ , where  $W$  is the weight,  $b$  is the bias, and  $A$  the ReLU activation function. The final layer uses  $A = Z$  (no activation).

Our chosen loss function is Mean Squared Error (MSE), that reflects how wrong the network's predictions are compared to the actual data. The loss function helps guide the gradient descent to adjust weights and biases to minimize MSE.

We used batch gradient descent to update weights using backpropagation. Additionally, we computed a "tolerance" hyperparameter variable that makes the training stop if the improvement in loss is smaller than the set tolerance value.

### 3.2.3 Hyperparameter Tuning

We explored through hyperparameters manually, trying different values until the RMSE couldn't get any lower. We adjusted the hidden layer size to 6 neurons, as we figured that it would be a neuron for each input feature, and it gave us the lowest RMSE between the 6, 16, and 32 neurons. We tried different learning rates such as 0.1, 0.01, 0.001, as well as many different max\_iterations. Ultimately, the choice combination that stood out was:

- Hidden layer size = 6 neurons
- Learning rate = 0.1
- max\_iterations = 100

- tolerance = 1e-4

### 3.2.4 Correctness Check

To check correctness, we trained the MLPRegressor reference model from scikit-learn. We tuned it to the same hyperparameters as our from-scratch model, used the same standardized numeric features, and the same target. We compared the training and test RMSE and overall behavior of the models, and since the results were almost identical to each other, we concluded that our model works accordingly.

## 3.3 Poisson Regression (M3)

### 3.3.1 Model Setup

Poisson regression is an algorithm commonly used for claim modeling and given that our response variable is a non-negative count, we believe it is a good fit for our project. The data we used to train the model consists of the six numeric features of our dataset (Exposure, VehPower, VehAge, DrivAge, BonusMalus, Density) as well as one categorical value (Area) that we encoded into integers (Area\_enc). These features were later standardized; however, we kept Area\_enc as is.

To handle different exposure lengths (due to some customers observed for shorter than 1 year), we trained our model using the target = ClaimNB / Exposure, and made Exposure our sample\_weight, as this weights policies with more “time at risk” more strongly.

We used the PoissonRegressor model from the scikit-learn library with two hyperparameters: alpha, and max iterations. We then measure RMSE on the model’s predicted claim frequency.

### 3.3.2 Hyperparameters

Our alpha parameter is used to control regularization, meaning that it adds a set penalty to the model that stops coefficients from becoming too large. We set the alpha to be 1e-4, which is relatively small, since we did not want our coefficients to be extremely shrunk, making our model more flexible.

The max\_iter parameter was set to 300, so that our model has a stopping limit of iterations in case convergence (result changes becoming small towards insignificant with each iteration) hasn't been achieved by then. This makes our model less computationally expensive, though, on average, the model shouldn't require all 300 iterations to reach its conclusion.

### 3.3.3 Correctness Check

We deduced the correctness of our model by following a few steps:

- We checked whether our predicted rates were non-negative, as the Poisson regression does not, and should not, by design, output negative rates.
- Made sure our model runs without any convergence warnings.
- Ensure the model predicts the right target variable.

## 4. Results and Discussion

This section compares the performance of the three modeling approaches used in this project: a Decision Tree Regressor, a Feedforward Neural Network, and a Poisson Regression model. All models were trained to predict **claim frequency**, defined as the number of claims divided by exposure, and evaluated using **RMSE** on both training and test data. In addition to RMSE, **calibration plots** were used to assess how well predicted claim frequencies align with observed outcomes across different risk levels.

### 4.1 Overall Performance Comparison

Table 1 summarizes the RMSE results for all three models.

|                     |               |              |
|---------------------|---------------|--------------|
| Decision Tree RMSE  | Train: 4.3471 | Test: 5.0629 |
| Feedforward NN RMSE | Train: 4.4244 | Test: 5.2015 |
| Poisson RMSE        | Train: 4.4289 | Test: 5.2061 |

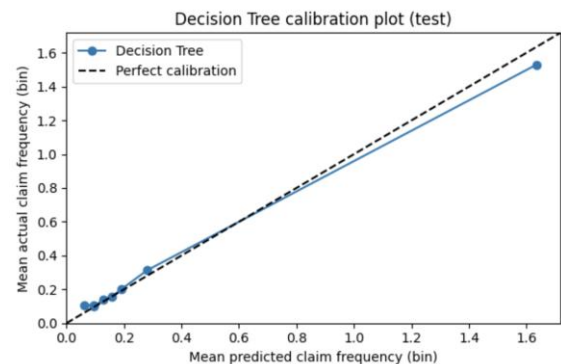
Among the three methods, the **Decision Tree achieved the lowest RMSE on both training and test sets**, indicating the best overall predictive performance. The Poisson Regression and Neural Network models produced very similar RMSE values, both slightly worse than the Decision Tree.

The gap between training and test RMSE is moderate and comparable across models, suggesting that none of the methods suffer from severe overfitting. However, differences become clearer when calibration and model behavior are examined more closely.

### 4.2 Decision Tree Model

The Decision Tree performs best overall, which is consistent with the nature of the insurance data. Claim frequency depends on **nonlinear relationships and interactions** between features such as driver age, bonus–malus score, exposure, and density. Decision trees are well suited to capture these effects because they split the feature space into regions with different average risks.

This behavior is also visible in the **calibration plot**. The decision tree shows a nearly linear relationship between mean predicted and mean observed claim frequency across prediction deciles, closely following the perfect calibration line. This indicates that the model not only predicts reasonable values on average but also **ranks policies by risk accurately**. The slight underestimation in the highest-risk bin likely reflects the scarcity of high-frequency claims in the data. Since extreme claim frequencies are rare, the tree tends to smooth predictions in those regions.

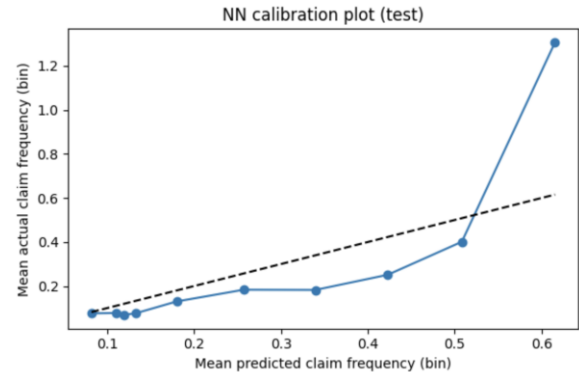


Overall, the Decision Tree provides a strong balance between flexibility, performance, and interpretability, making it the best-performing model in our project.

### 4.3 Feedforward Neural Network

The Neural Network performs slightly worse than the Decision Tree, with the highest test RMSE among the three methods. While the network is capable of modeling nonlinear relationships, several factors limit its effectiveness in this setting.

First, the network uses **mean squared error** as its loss function, which is not ideal for heavily zero-inflated count data. Most policies have zero claims, and MSE places disproportionate emphasis on rare large errors. Second, exposure is not modeled explicitly as an offset, meaning the network must implicitly learn how exposure affects risk, which is difficult given the limited architecture.



This behavior is clearly visible in the **neural network calibration plot**. While predictions increase with observed risk, the curve deviates substantially from the perfect calibration line, especially at higher predicted values. This indicates that the network can roughly order policies by risk but struggles to produce well-scaled predictions.

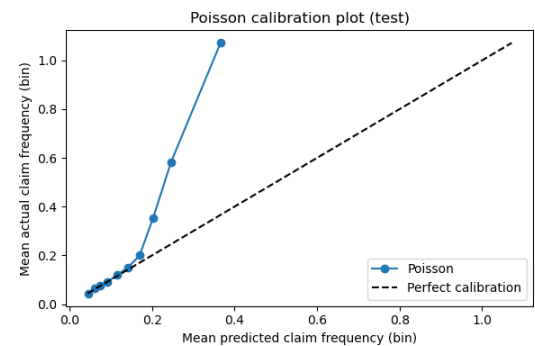
In summary, the neural network shows reasonable learning behavior but is **not well matched to the statistical structure of insurance claim data**, especially given its simple architecture and loss function.

### 4.4 Poisson Regression

Poisson Regression produces performance similar to the neural network, slightly worse than the decision tree but still competitive. This is notable given the simplicity of the model.

Unlike the neural network, Poisson regression is **designed specifically for count data** and explicitly accounts for exposure by modeling claim frequency with exposure-based weighting. This strong statistical assumption helps stabilize predictions, particularly in low-risk regions.

The calibration plot shows that the Poisson model is well calibrated at low and medium predicted claim frequencies, where most policies are located. In this region, the predictions closely follow the perfect calibration line, highlighting the model's strength in handling exposure and modeling claim frequency in a statistically sound way. At higher predicted frequencies, the model tends to underpredict the actual claim frequency, visible as a deviation above the diagonal line.



This behavior is expected given the log-linear structure of Poisson regression, which limits its ability to capture strong nonlinear effects in high-risk segments.

Despite these limitations, Poisson regression remains viable due to its **interpretability and robustness**, especially in contexts where transparency is important.

## 5. Conclusion

In conclusion, the comparison of the three models shows that model choice must match the structure of the data. The decision tree achieved the best overall performance, as it can naturally capture nonlinear effects and interactions between variables. Poisson regression performed slightly worse but remained competitive due to its principled treatment of exposure and good calibration for most policies, despite its limited flexibility. The neural network showed a similar performance to the Poisson regressor; however, due to its simple architecture, lack of explicit exposure modeling, and the use of the MSE loss function, it is not well suited for the data that was fed into it.

Overall, the results confirm that more complex models do not necessarily perform better. In this project, the decision tree provides the best balance between accuracy, flexibility, and calibration, while Poisson regression remains a strong and interpretable baseline.

## Use of generative AI

The work above was assisted by the Generative AI tool, ChatGPT. This tool was used to better understand the concepts presented in lectures and base code structure, as well as reword parts of the report in an academic manner. For example, we used the tool to clarify questions such as: “What’s the standard hyperparameters for the base model of Poisson regression?”, “What’s the best plot to visualize region density?”, “Make my paragraph shorter, by eliminating irrelevant or repeated information”.

## References

- [1] [https://thesai.org/Downloads/Volume14No9/Paper\\_66-AIRA\\_ML\\_Auto\\_Insurance\\_Risk\\_Assessment\\_Machine\\_Learning\\_Model.pdf](https://thesai.org/Downloads/Volume14No9/Paper_66-AIRA_ML_Auto_Insurance_Risk_Assessment_Machine_Learning_Model.pdf)
- [2] <https://safetyfirstinsure.com/modeling-claim-frequency-and-severity/>
- [3] <https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/>
- [4] <https://github.com/matsjfunke/feedforward-neural-network-from-scratch>